

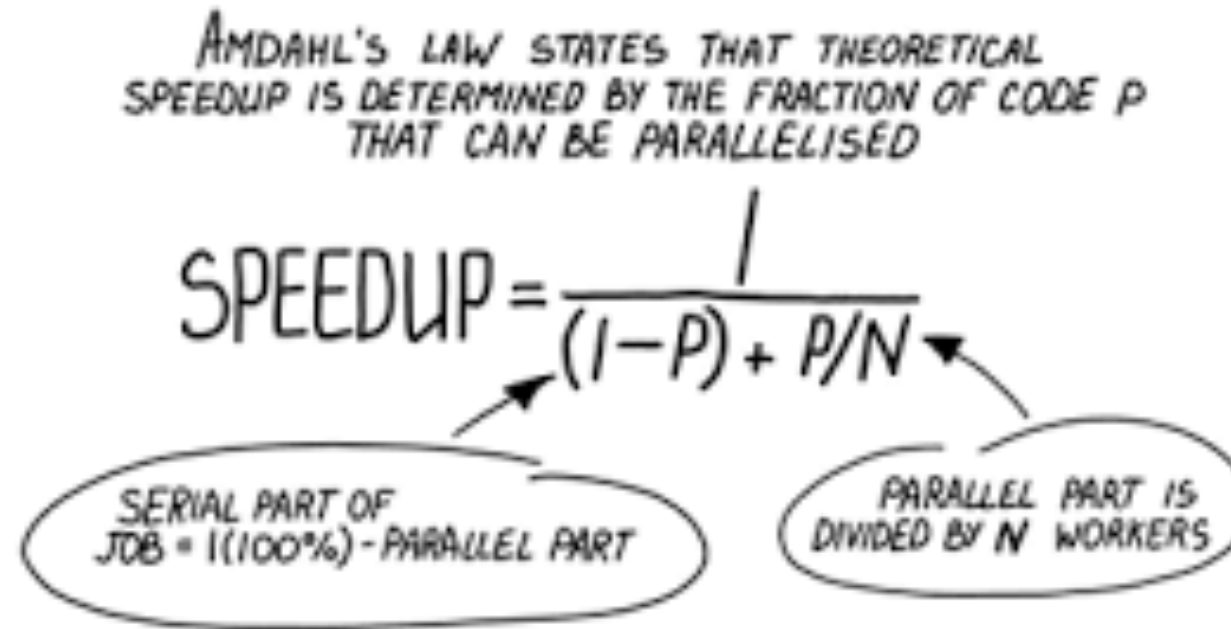


Profiling in MPI

2026

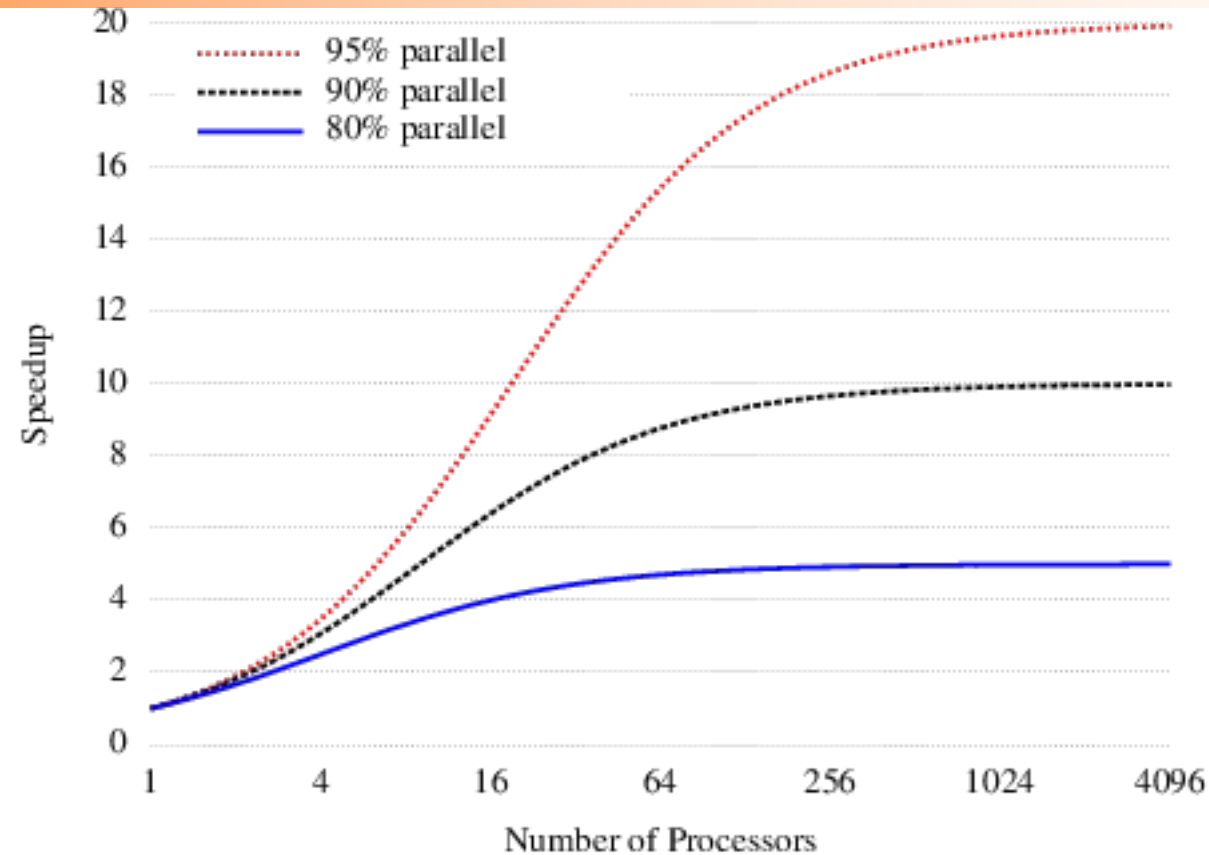
André Pereira

Theoretical Performance Boundaries



- Is it possible to have superlinear speedups?
 - Does it model common bottlenecks?

Theoretical Performance Boundaries

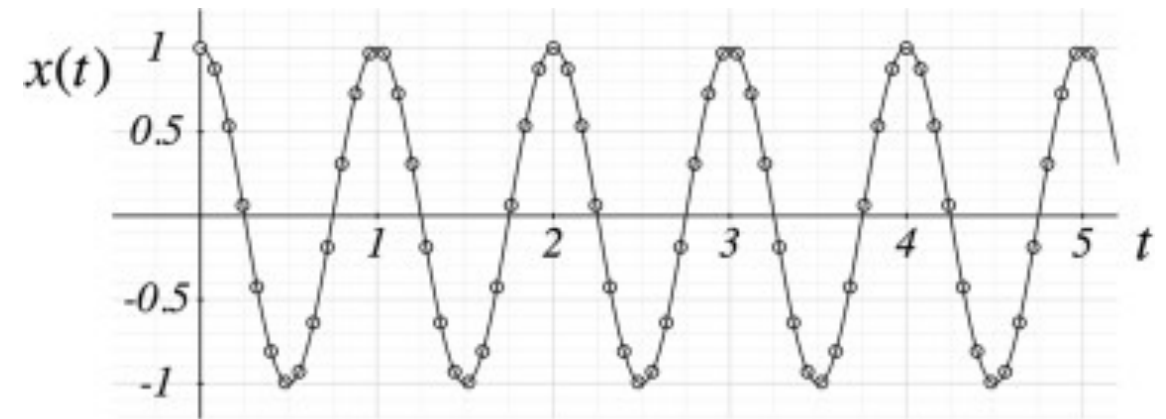


- What component changes to account for communication overheads?

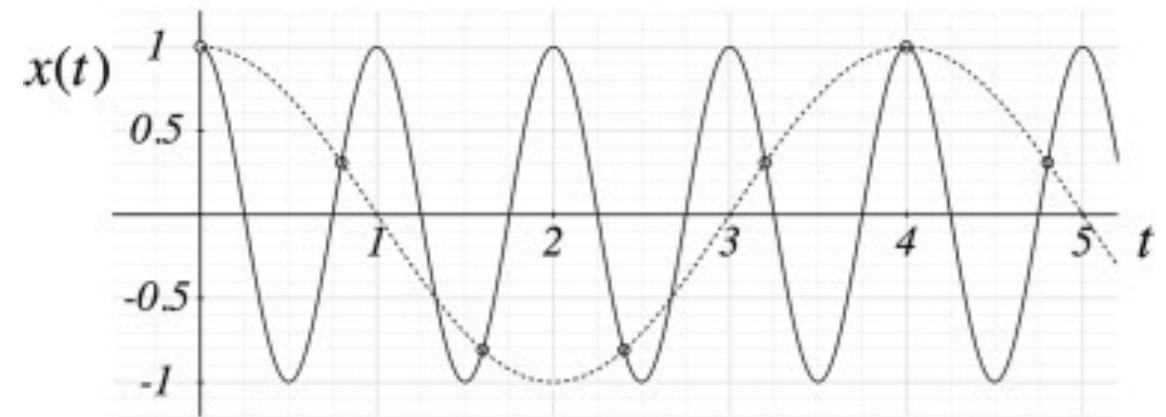


Statistical Sampling

- Periodically interrupts the processor to record performance counters and call stack
- It yields low, deterministic overhead with minimal perturbation of application behaviour
- It provides a statistical approximation that may obscure high-frequency events



(A)



(B)



overhead == 😞

no overhead == 😊

Event-Based Instrumentation

- Injects precise measurement hooks directly into the application binary
- Guarantees exact execution counts and deterministic traces
- Introduces severe overhead and risks the "Observer Effect," altering timing characteristics



Statistical Sampling

- Periodically interrupts the processor to record performance counters and call stack
- It yields low, deterministic overhead with minimal perturbation of application behaviour
- It provides a statistical approximation that may obscure high-frequency events

Event-Based Instrumentation

- Injects precise measurement hooks directly into the application binary
- Guarantees exact execution counts and deterministic traces
- Introduces severe overhead and risks the "Observer Effect," altering timing characteristics

Multithreading vs MPI – Profiling Implications



ARCHITECTURAL TRAIT	MULTITHREADING	MESSAGE PASSING (MPI)
Memory Space	Global address space (UMA/NUMA architectures)	Independent compute nodes connected via network fabric
Profiling Focus	Cache coherency, lock contention, false sharing	Interconnect latency, bandwidth saturation, message serialisation
Synchronization	Unified system clock makes event ordering trivial	Requires complex cross-node clock synchronization algorithms

Complexities of Profiling MPI Code – Causal Integrity



Clock Drift and Skew

- Compute nodes maintain their own hardware oscillators
- Over the lifespan of an application, these oscillators drift
- Aggressive synchronisation via the Precision Time Protocol (PTP)
 - timestamps across a cluster rapidly diverge, rendering chronological analysis impossible

Causal Paradoxes

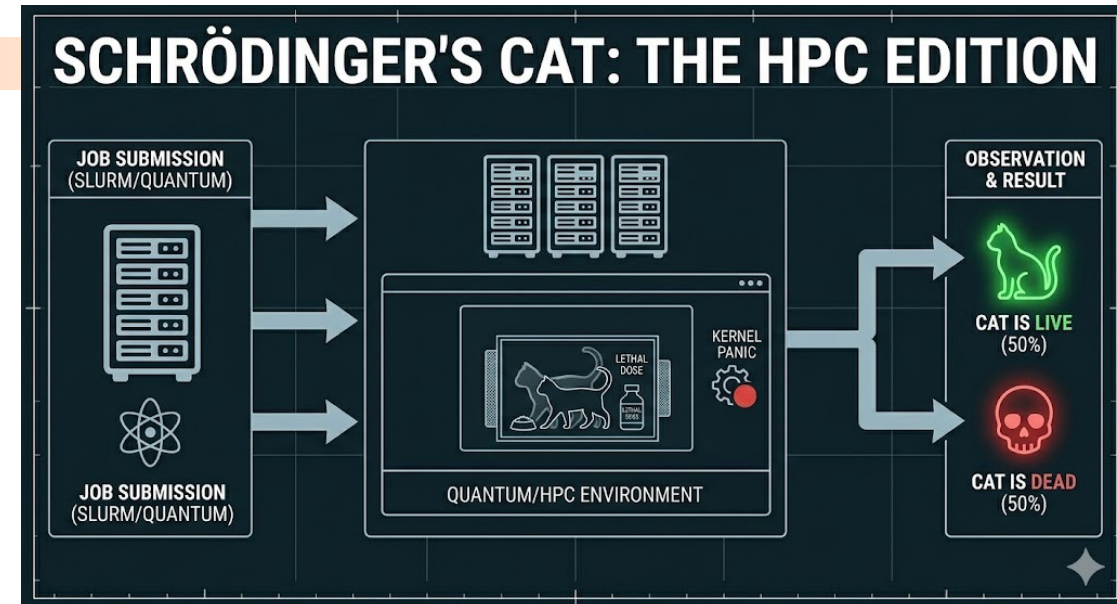
- Assume Node A's clock is 500 microseconds behind Node B's
 - A message sent from A to B might be logged as arriving **before** it was sent
- Profilers implement complex Lamport logical clocks or rigorous *post-mortem* trace adjustments
 - prevent these causal violations in the analysis
 - This is a topic for Distributed Systems

Complexities of Profiling MPI Code – Perturbation and Data Deluge



- The Observer Effect

- The act of measuring alters the system.
- Inserting `MPI_Wtime` calls interrupts instruction pipelines and pollutes L1/L2 caches
 - Artificially altering the execution timeline
 - This perturbation can mask real bottlenecks or create phantom ones



Complexities of Profiling MPI Code – Perturbation and Data Deluge



- The amount of tracing data
 - Instrumenting point-to-point operations on 100,000+ ranks generates terabytes of raw chronological data per minute
 - Unmanaged, this I/O storm will catastrophically crash parallel file systems (like Lustre or GPFS)

Complexities of Profiling MPI Code – Perturbation and Data Deluge



- The mitigation strategies
 - Modern profilers utilise the Open Trace Format 2 (OTF2) for highly compressed, massively parallel I/O
 - Rely heavily on selective rank filtering, time-windowing, and call-path statistical sampling to scale

Complexities of Profiling MPI Code – Protocol Overheads



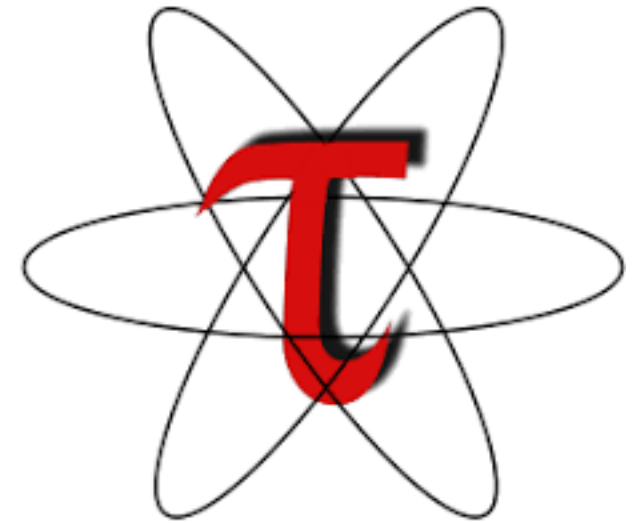
- Differentiating network hardware from software protocol overhead is complex
 - MPI libraries utilise distinct transfer protocols dynamically based on message size.
- **Eager Protocols**
 - Send small data immediately to pre-allocated receiver buffers
 - Maximises bandwidth but risks memory exhaustion
- **Rendezvous Protocols**
 - Require a control-message handshake before transferring large payloads
 - Secure memory safety, but increase baseline latency
 - Profilers must isolate these distinct software states from physical interconnect delays



HPC Profiling Frameworks



- TAU is a robust, highly configurable suite for source-level and binary performance instrumentation
 - It excels at measuring heterogeneous computing environments (MPI, OpenMP, CUDA, ROCm) concurrently
- **Program Database Toolkit (PDT)**
 - TAU utilises PDT to parse source code structurally
 - Enables automated, precise insertion of measurement hooks





- **Multi-Level Profiling**
 - Generates both inclusive and exclusive flat profiles
 - Generates deeply nested chronological event traces
- **Compiler Integration**
 - Supports dynamic intercept via LD_PRELOAD and PMPI
 - Supports static compiler-level wrapper instrumentation for absolute control over the observer effect



- **Asynchronous Statistical Sampling**

- HPCToolkit eschews code modification
- Utilises timer and hardware performance counter interrupts to statistically sample the execution state

- **Call-Path Unwinding**

- At each interrupt, HPCToolkit captures and unwinds the active call stack
- It builds a highly accurate, hierarchical tree of execution attribution
 - Mapping performance metrics directly to specific source code loops and inline functions



HPCToolkit



- **Asynchronous Statistical Sampling**
- **Call-Path Unwinding**
- **Observer Effect Mitigation**
 - Sampling-based, analyses fully optimised binaries without recompilation
 - Measurement overhead remains consistently low (typically 1-5%), ensuring the measured execution mirrors production reality



- Score-P was developed to consolidate several independent, overlapping tracing tools
 - Like VampirTrace and early Scalasca monitors
- It natively orchestrates multi-paradigm instrumentation
- Seamless aggregation of PMPI, OMPT (OpenMP Tools Interface), and CUDA/CUPTI telemetry





- Score-P does not visualise data itself
 - It acts as the high-throughput telemetry engine
 - It outputs data into two rigidly standardized formats
- **CUBE4**
 - A highly compressed, multi-dimensional data format for summarised profiling
- **OTF2 (Open Trace Format 2)**
 - A massively parallel, highly scalable chronological trace format
 - Designed to support I/O generation across 100,000+ ranks without saturating parallel file systems

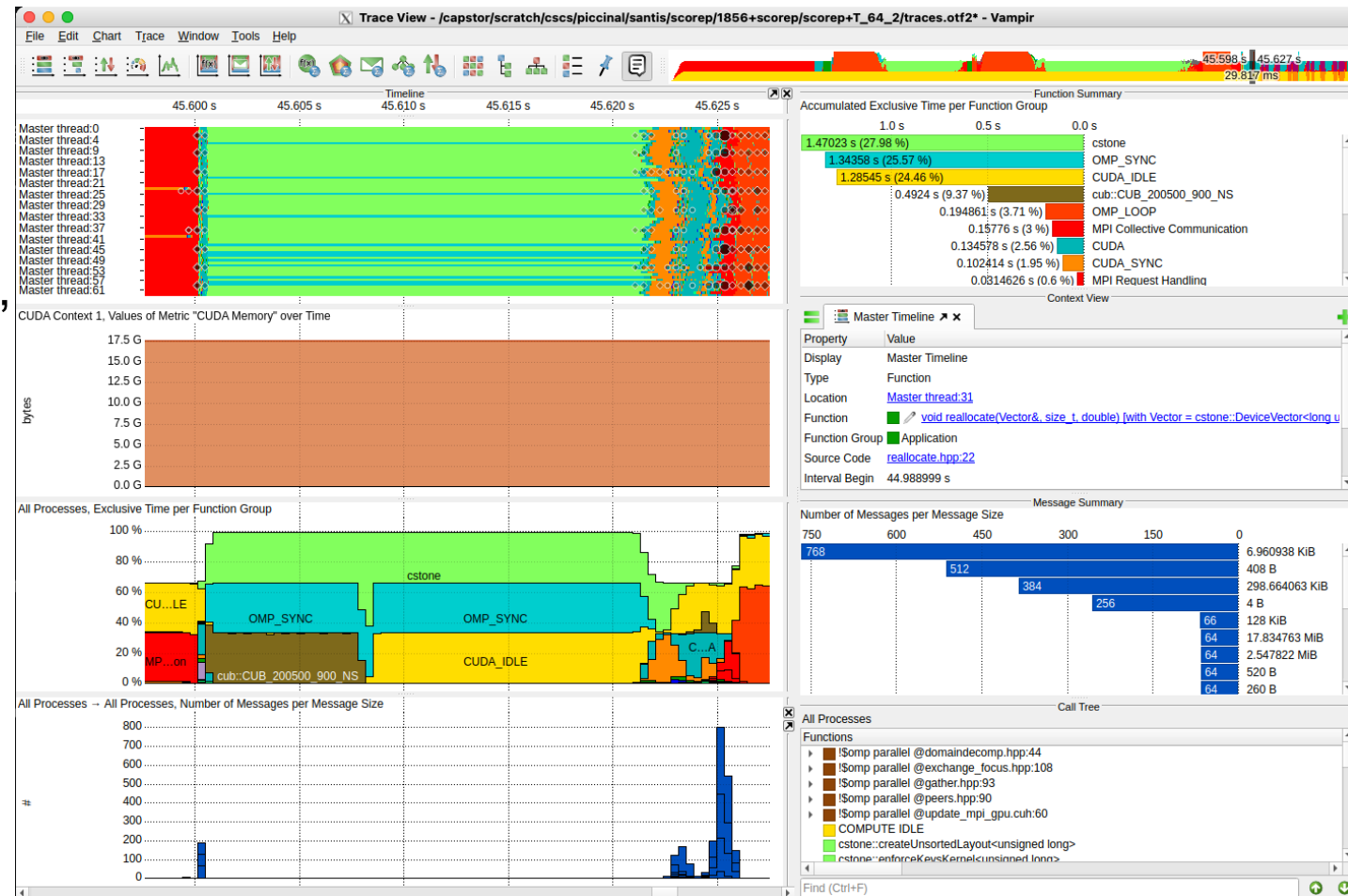


- Scalasca operates on the CUBE4 profiles and OTF2 traces generated by Score-P.
- **Parallel Trace Replay**
 - Scalasca algorithms replay the OTF2 trace chronologically in parallel across the compute cluster
- **Heuristic Identification**
 - It automatically isolates and quantifies specific communication inefficiencies
 - Calculates delays caused by "Late Senders," "Late Receivers," and cascading `MPI_Wait` blockages

Visualising – Scalasca



- Parallel Trace Replay
- Heuristic Identification
- Topological Delay Attribution
 - It maps logical communication delays to the physical network topology, identifying sub-optimal algorithmic scaling limits





- **Interactive Trace Visualisation**
 - Vampir allows deep, interactive exploration of OTF2 trace datasets
- **Granular Chronological Navigation**
 - Analysis of macroscopic, application-wide timeline covering thousands of MPI ranks, down to microsecond-level hardware events on a single CUDA stream
- **Matrix & Flow Analytics**
 - Graphical representation of process communication matrices
 - Highlights skewed data exchange patterns
 - Identifies point-to-point network fabric congestion



Nsight Systems `nsys`

- Nsight Systems maps the entire system execution on a unified chronological timeline
- Its primary objective is identifying temporal gaps where the GPU is starved
- Tracks CPU thread scheduling, MPI communication bottlenecks, and API synchronisation overheads

Nsight Compute `ncu`

- The micro-level kernel profiler
- Used strictly *after* identifying compute-bound bottlenecks in `nsys`
- Focuses on deep hardware metric collection for individual CUDA kernels
 - Cache hit rates, register pressure, memory throughput, and warp divergence



- Nsight captures the state of every OS thread
 - Running, Sleeping, Blocked, or Preempted
- This telemetry is critical for diagnosing thread anomalies
 - Mutex lock contention, false sharing, and inefficient barrier synchronisations
- Maps CPU thread activity to hardware core migrations
 - Exposes OS scheduling jitter and NUMA affinity issues
- Correlates multi-core CPU execution with asynchronous CUDA API calls

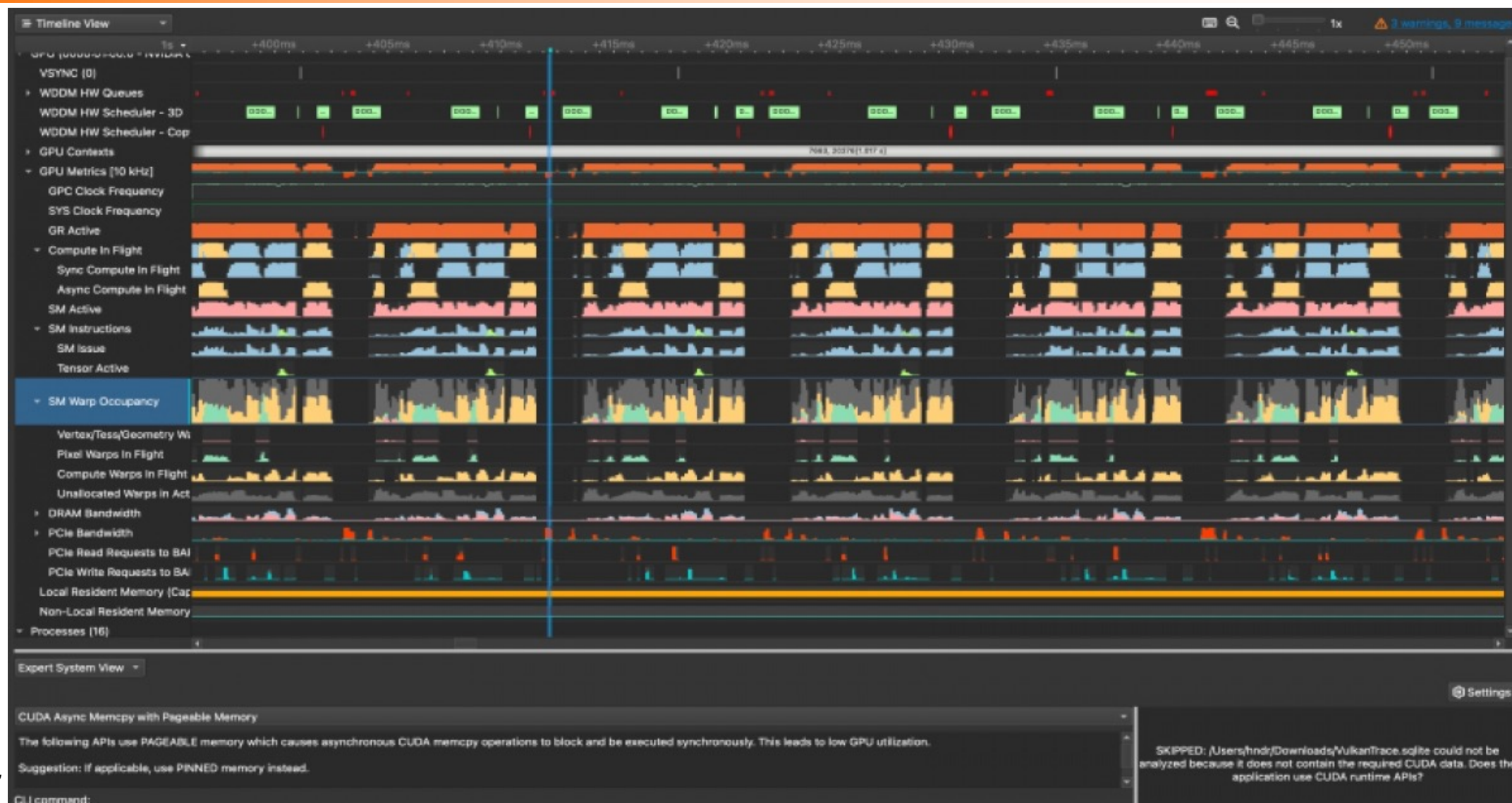


- **PMPI Interception**
 - Transparently leverages the PMPI standard interface to log MPI calls without requiring application recompilation
- **Protocol Overhead**
 - Distinguishes between the CPU cycles spent executing the MPI library and the actual network transfer latency
- **Visualizing Serialization**
 - Exposes node-to-node communication blockages
 - Cascading collective synchronizations and inefficient overlap of non-blocking I/O with background computation



- Tracks all cudaMemcpy events alongside kernel execution durations
 - Visualises concurrent CUDA Streams, allowing verification that multiple independent kernels are overlapping execution on the GPU to hide latency
- Captures Unified Virtual Memory page faults and GPUDirect RDMA transactions
 - Allows to verify if NICs are successfully bypassing the host CPU to write MPI message payloads directly into GPU VRAM

Nsight - Visualisation



Nsight - Visualisation

